

Seance 6

Exceptions et Gestion d'Erreurs
Gerer les erreurs proprement et en securite

Cours de renforcement C++

Table des matières

1	Introduction : Le probleme de la gestion d'erreurs	3
1.1	Rappel des seances precedentes	3
1.2	Les anciennes methodes (C)	3
1.3	La solution moderne : Les exceptions	4
2	Syntaxe de base : try, throw, catch	5
2.1	throw : Lancer une exception	5
2.2	try-catch : Attraper une exception	5
2.3	Le flux d'execution	6
2.4	Plusieurs catch	6
2.5	catch(...) : Attraper TOUT	7
3	La hierarchie std : :exception	8
3.1	La classe de base	8
3.2	Les exceptions logiques (logic_error)	8
3.3	Les exceptions runtime (runtime_error)	8
4	Creer ses propres exceptions	10
4.1	Heriter de std : :exception	10
4.2	Hierarchie d'exceptions personnalisees	10
4.3	Ajouter des informations supplementaires	11
5	Exception safety : Garanties de securite	13
5.1	Les trois niveaux de garantie	13
5.2	RAII et exception safety	13
5.3	Destructeurs et exceptions	14
6	Stack unwinding : Deroulement de la pile	15
6.1	Le concept	15
6.2	Exemple avec ressources multiples	16
7	Relancer une exception	17
7.1	throw ; sans argument	17
7.2	Wrapper une exception	17
8	Exercices pratiques	18
8.1	Exercice 1 : Validation avec exceptions (facile)	18
8.2	Exercice 2 : Exception personnalisee (moyen)	19
8.3	Exercice 3 : RAII avec exceptions (difficile)	20

9 Patterns avec exceptions	22
9.1 Exception Guard	22
9.2 Try-Catch dans les constructeurs	22
9.3 Error Codes vs Exceptions	23
10 Pieges et bonnes pratiques	24
10.1 Pieu 1 : Catch par valeur	24
10.2 Pieu 2 : Lancer par pointeur	24
10.3 Bonne pratique 1 : Exception dans les templates	24
10.4 Bonne pratique 2 : noexcept quand garanti	24
11 Quiz de validation	26
11.1 Questions	26
11.2 Reponses	26

1 Introduction : Le probleme de la gestion d'erreurs

1.1 Rappel des seances precedentes

Dans la Seance 5, nous avons cree des templates generiques. Voici un exemple d'une classe `Array<T>` :

```

1  template<typename T>
2  class Array {
3  private:
4      T* data;
5      size_t taille;
6  public:
7      T& operator[](size_t index) {
8          if (index >= taille) {
9              // QUE FAIRE EN CAS D'ERREUR ?
10             // 1. Retourner une valeur par default ? Mais laquelle ?
11             // 2. Retourner un code d'erreur ? Impossible, on doit
12                retourner T&
13             // 3. Crasher ? Trop brutal !
14         }
15         return data[index];
16     }
17 };

```

Le probleme

Comment signaler une erreur quand :

- On ne peut pas retourner de code d'erreur (fonction retourne autre chose)
- Il n'y a pas de valeur "erreur" logique a retourner
- L'erreur est critique et doit etre traitee
- L'erreur peut se produire au milieu d'une longue chaine d'operations

1.2 Les anciennes methodes (C)

Methode 1 : Codes d'erreur

```

1  int diviser(int a, int b, int* resultat) {
2      if (b == 0) {
3          return -1; // Code d'erreur
4      }
5      *resultat = a / b;
6      return 0; // Succes
7  }
8
9  int main() {
10     int resultat;
11     if (diviser(10, 0, &resultat) != 0) {
12         std::cout << "Erreur !\n";
13     }
14     // Probleme : facile d'oublier de verifier le code de retour !
15 }

```

Methode 2 : Variable globale errno

```

1  #include <cerrno>
2

```

```

3 FILE* fichier = fopen("inexistant.txt", "r");
4 if (fichier == NULL) {
5     std::cout << "Erreur : " << errno << "\n";
6 }
7 // Probleme : errno est global, pas thread-safe !

```

Pourquoi c'est mauvais

Codes d'erreur :

- Facile d'oublier de verifier
- Pollue la signature des fonctions
- Ne peut pas traverser plusieurs niveaux de fonctions facilement

errno :

- Variable globale (dangereux en multi-threading)
- Doit etre verifie immediatement
- Codes d'erreur obscurs

1.3 La solution moderne : Les exceptions

Une exception est un mecanisme pour signaler et gerer les erreurs qui :

1. Interrompt le flux d'execution normal
2. Remonte automatiquement la pile d'appels
3. Peut etre attrapee et traitee a n'importe quel niveau
4. Force le traitement (le programme crash si non geree)

```

1 template<typename T>
2 T& Array<T>::operator[](size_t index) {
3     if (index >= taille) {
4         throw std::out_of_range("Index hors limites");
5     }
6     return data[index];
7 }
8
9 // Utilisation
10 try {
11     Array<int> arr(5);
12     std::cout << arr[10] << "\n"; // ERREUR !
13 } catch (const std::out_of_range& e) {
14     std::cout << "Erreur attrapee : " << e.what() << "\n";
15 }

```

Les avantages

- **Separation** : Le code "normal" est separe du code de gestion d'erreurs
- **Impossible d'ignorer** : Une exception non geree crash le programme
- **Propagation automatique** : L'erreur remonte jusqu'a etre attrapee
- **Information riche** : On peut passer des objets complets, pas juste des codes

2 Syntaxe de base : try, throw, catch

2.1 throw : Lancer une exception

```
1 void verifierAge(int age) {
2     if (age < 0) {
3         throw std::invalid_argument("L'age ne peut pas etre negatif");
4     }
5     if (age > 150) {
6         throw std::invalid_argument("L'age est trop grand");
7     }
8     std::cout << "Age valide : " << age << "\n";
9 }
```

throw dit : "STOP! Il y a une erreur! Je remonte ca a celui qui m'a appele."

2.2 try-catch : Attraper une exception

```
1 int main() {
2     try {
3         // Code qui peut lancer une exception
4         verifierAge(200);
5         std::cout << "Cette ligne ne sera jamais executee\n";
6     } catch (const std::invalid_argument& e) {
7         // Code qui traite l'erreur
8         std::cout << "Erreur attrapee : " << e.what() << "\n";
9     }
10
11     std::cout << "Le programme continue\n";
12 }
13
14 // Affiche :
15 // Erreur attrapee : L'age est trop grand
16 // Le programme continue
```

Decomposition

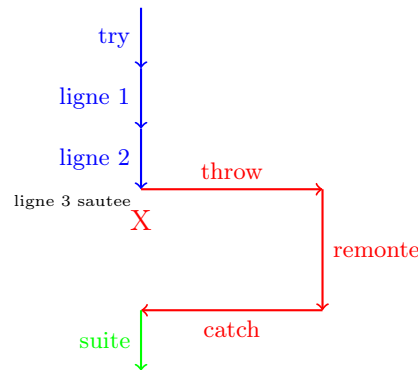
try { ... } : "Essaie d'exécuter ce code. Si une exception est lancée, arrête et cherche un catch."

catch (const Type& e) { ... } : "Si une exception de type Type est lancée, exécute ce code pour la traiter."

e.what() : Méthode qui retourne le message d'erreur (toutes les exceptions standard ont cette méthode)

2.3 Le flux d'exécution

Flux avec exception



Quand une exception est lancée :

1. Le reste du bloc try est SAUTE
2. On remonte la pile d'appels en cherchant un catch
3. Quand un catch correspondant est trouvé, on exécute son code
4. Puis on continue après le bloc try-catch

2.4 Plusieurs catch

```

1 void traiterFichier(const std::string& nom) {
2     try {
3         std::ifstream fichier(nom);
4         if (!fichier) {
5             throw std::runtime_error("Impossible d'ouvrir le fichier");
6         }
7
8         int nombre;
9         fichier >> nombre;
10        if (fichier.fail()) {
11            throw std::invalid_argument("Format invalide");
12        }
13
14        if (nombre < 0) {
15            throw std::out_of_range("Nombre negatif");
16        }
17
18        std::cout << "Nombre lu : " << nombre << "\n";
19
20    } catch (const std::runtime_error& e) {
21        std::cout << "Erreur runtime : " << e.what() << "\n";
22    } catch (const std::invalid_argument& e) {
23        std::cout << "Argument invalide : " << e.what() << "\n";
24    } catch (const std::out_of_range& e) {
25        std::cout << "Hors limites : " << e.what() << "\n";
26    }
27 }
  
```

Le premier catch qui correspond est exécuté. Les autres sont ignorés.

2.5 catch(...) : Attraper TOUT

```
1 try {  
2     // Code dangereux...  
3 } catch (const std::exception& e) {  
4     // Attrape toutes les exceptions standard  
5     std::cout << "Exception : " << e.what() << "\n";  
6 } catch (...) {  
7     // Attrape ABSOLUMENT TOUT (meme les exceptions non-standard)  
8     std::cout << "Exception inconnue !\n";  
9 }
```

Ordre des catch

IMPORTANT : Mettez les catch du plus specifique au plus general !

```
1 // BON  
2 catch (const std::out_of_range& e) { /* ... */ }  
3 catch (const std::exception& e) { /* ... */ }  
4 catch (...) { /* ... */ }  
5  
6 // MAUVAIS  
7 catch (...) { /* ... */ }  
8 catch (const std::exception& e) { /* ... */ } // Jamais execute !
```

Pourquoi? Le premier catch qui correspond est pris. Si vous mettez `catch(...)` en premier, il attrape TOUT et les autres ne sont jamais executes.

3 La hierarchie std::exception

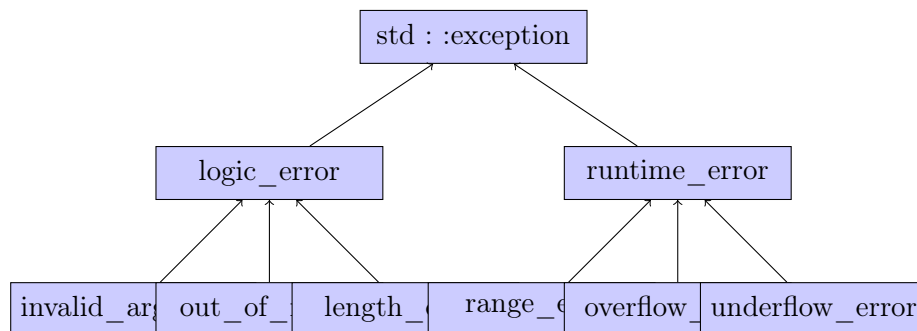
3.1 La classe de base

Toutes les exceptions standard heritent de `std::exception` :

```

1 class exception {
2 public:
3     virtual const char* what() const noexcept = 0;
4     virtual ~exception() = default;
5 };

```



3.2 Les exceptions logiques (logic_error)

Erreurs de logique du programme (bugs) :

```

1 // invalid_argument : Argument invalide
2 void setAge(int age) {
3     if (age < 0 || age > 150) {
4         throw std::invalid_argument("Age invalide");
5     }
6 }
7
8 // out_of_range : Index hors limites
9 std::vector<int> vec = {1, 2, 3};
10 std::cout << vec.at(10) << "\n"; // Throw out_of_range
11
12 // length_error : Taille trop grande
13 std::string s;
14 s.resize(s.max_size() + 1); // Throw length_error

```

3.3 Les exceptions runtime (runtime_error)

Erreurs qui ne peuvent etre detectees qu'a l'execution :

```

1 // runtime_error : Erreur generale
2 if (!fichier.is_open()) {
3     throw std::runtime_error("Impossible d'ouvrir le fichier");
4 }
5
6 // overflow_error : Depassement de capacite
7 // underflow_error : Sous-depassement
8
9 // range_error : Valeur hors de la plage valide

```

Quelle exception choisir

logic_error : Erreur de programmation, bug (devrait etre evite par le code)

runtime_error : Erreur liee a l'environnement (fichiers, reseau, entrees utilisateur)

En pratique : Si vous hesitez, utilisez **runtime_error**. C'est le choix le plus general.

4 Créer ses propres exceptions

4.1 Heriter de std::exception

```

1 class MonException : public std::exception {
2 private:
3     std::string message;
4
5 public:
6     MonException(const std::string& msg) : message(msg) {}
7
8     const char* what() const noexcept override {
9         return message.c_str();
10    }
11 };
12
13 void fonction() {
14     throw MonException("Quelque chose s'est mal passe");
15 }
16
17 int main() {
18     try {
19         fonction();
20     } catch (const MonException& e) {
21         std::cout << "Mon exception : " << e.what() << "\n";
22     } catch (const std::exception& e) {
23         std::cout << "Exception standard : " << e.what() << "\n";
24     }
25 }

```

Pourquoi noexcept ?

const char* what() const noexcept

noexcept signifie que cette fonction ne lancera JAMAIS d'exception.

C'est important pour what() car si elle lançait une exception pendant le traitement d'une autre exception, le programme crasherait immédiatement (double exception).

4.2 Hierarchie d'exceptions personnalisées

```

1 // Exception de base pour notre application
2 class AppException : public std::exception {
3 protected:
4     std::string message;
5 public:
6     AppException(const std::string& msg) : message(msg) {}
7     const char* what() const noexcept override {
8         return message.c_str();
9     }
10 };
11
12 // Exceptions spécifiques
13 class DatabaseException : public AppException {
14 public:
15     DatabaseException(const std::string& msg)
16         : AppException("DB Error: " + msg) {}
17 };

```

```

18
19 class NetworkException : public AppException {
20 public:
21     NetworkException(const std::string& msg)
22         : AppException("Network Error: " + msg) {}
23 };
24
25 class ValidationException : public AppException {
26 public:
27     ValidationException(const std::string& msg)
28         : AppException("Validation Error: " + msg) {}
29 };

```

Utilisation :

```

1 try {
2     // Code qui peut lancer différentes exceptions
3     if (connexionPerdue) {
4         throw NetworkException("Connexion perdue");
5     }
6     if (donneesInvalides) {
7         throw ValidationException("Email invalide");
8     }
9
10 } catch (const DatabaseException& e) {
11     std::cout << "Probleme de base de donnees\n";
12 } catch (const NetworkException& e) {
13     std::cout << "Probleme reseau\n";
14 } catch (const AppException& e) {
15     std::cout << "Erreur application : " << e.what() << "\n";
16 }

```

4.3 Ajouter des informations supplémentaires

```

1 class FileException : public std::exception {
2 private:
3     std::string filename;
4     std::string operation;
5     std::string message;
6
7 public:
8     FileException(const std::string& file,
9                  const std::string& op,
10                 const std::string& msg)
11         : filename(file), operation(op) {
12         message = "Fichier '" + filename + "': " +
13                 "Operation '" + operation + "' - " + msg;
14     }
15
16     const char* what() const noexcept override {
17         return message.c_str();
18     }
19
20     const std::string& getFilename() const { return filename; }
21     const std::string& getOperation() const { return operation; }
22 };
23
24 // Utilisation

```

```
25 try {  
26     throw FileException("data.txt", "open", "Fichier introuvable");  
27 } catch (const FileException& e) {  
28     std::cout << e.what() << "\n";  
29     std::cout << "Fichier concerne : " << e.getFilename() << "\n";  
30 }
```

5 Exception safety : Garanties de securite

5.1 Les trois niveaux de garantie

1. Basic guarantee (garantie de base)

Le programme reste dans un etat valide. Pas de fuites memoire, pas de corruption.

```

1 void fonction() {
2     auto ptr = std::make_unique<int>(42);
3     // Quelque chose peut throw ici
4     traiterDonnees();
5     // Si exception : ptr est automatiquement libere (RAII)
6 }

```

2. Strong guarantee (garantie forte)

Si une exception est lancee, l'etat du programme est identique a avant l'appel. C'est comme si la fonction n'avait jamais ete appelee.

```

1 void Stack::push(int value) {
2     std::vector<int> temp = data; // Copie
3     temp.push_back(value);       // Peut throw
4     data = std::move(temp);      // Ne throw jamais (move)
5     // Si exception : data est inchangé
6 }

```

3. Nothrow guarantee (garantie de non-exception)

La fonction ne lance JAMAIS d'exception.

```

1 int Stack::size() const noexcept {
2     return data.size();
3 }
4
5 void swap(int& a, int& b) noexcept {
6     int temp = a;
7     a = b;
8     b = temp;
9 }

```

Quelle garantie viser ?

Basic : Minimum absolu. Toutes vos fonctions doivent au moins offrir ça.

Strong : Ideal quand c'est possible. Utilisez RAII et copy-and-swap.

Nothrow : Pour les fonctions critiques (destructeurs, swap, move). Marquez-les **noexcept**.

5.2 RAII et exception safety

RAII garantit automatiquement la basic guarantee :

```

1 void mauvaisCode() {
2     int* ptr = new int(42);
3     traiterDonnees(); // Peut throw !
4     delete ptr;       // Jamais execute si exception !
5     // FUITE MEMOIRE
6 }
7
8 void bonCode() {
9     auto ptr = std::make_unique<int>(42);
10    traiterDonnees(); // Peut throw

```

```
11 // ptr est automatiquement libere grace au destructeur
12 // AUCUNE FUITE
13 }
```

5.3 Destructeurs et exceptions

REGLE ABSOLUE

UN DESTRUCTEUR NE DOIT JAMAIS LANCER D'EXCEPTION !

```
1 class MauvaiseClasse {
2     ~MauvaiseClasse() {
3         throw std::runtime_error("Erreur"); // CATASTROPHIQUE !
4     }
5 };
```

Pourquoi ? Si une exception est déjà en cours (stack unwinding) et que le destructeur en lance une autre, le programme appelle `std::terminate()` et crash immédiatement.

Solution : Marquez tous vos destructeurs `noexcept` (c'est le défaut en C++11+) et gerez les erreurs autrement.

```
1 class BonneClasse {
2 public:
3     ~BonneClasse() noexcept { // Explicit ou implicite
4         try {
5             // Code qui peut throw
6             riskyOperation();
7         } catch (...) {
8             // Absorber l'exception, ne pas la relancer
9             std::cerr << "Erreur dans le destructeur\n";
10        }
11    }
12 };
```

6 Stack unwinding : Deroulement de la pile

6.1 Le concept

Quand une exception est lancee, C++ "deroule" la pile d'appels en appelant tous les destructeurs des objets locaux.

```
1 class Trace {
2 private:
3     std::string nom;
4 public:
5     Trace(const std::string& n) : nom(n) {
6         std::cout << nom << " construit\n";
7     }
8     ~Trace() {
9         std::cout << nom << " detruit\n";
10    }
11 };
12
13 void niveau3() {
14     Trace t("niveau3");
15     throw std::runtime_error("Erreur au niveau 3");
16 }
17
18 void niveau2() {
19     Trace t("niveau2");
20     niveau3();
21 }
22
23 void niveau1() {
24     Trace t("niveau1");
25     niveau2();
26 }
27
28 int main() {
29     try {
30         niveau1();
31     } catch (const std::exception& e) {
32         std::cout << "Exception attrapee : " << e.what() << "\n";
33     }
34 }
35
36 // Affiche :
37 // niveau1 construit
38 // niveau2 construit
39 // niveau3 construit
40 // niveau3 detruit <- Stack unwinding
41 // niveau2 detruit <- Stack unwinding
42 // niveau1 detruit <- Stack unwinding
43 // Exception attrapee : Erreur au niveau 3
```

Stack unwinding

Quand une exception est lancee :

1. On sort immediatement de la fonction actuelle
2. Tous les objets locaux sont detruits (destructeurs appeles)
3. On remonte a la fonction appelante
4. On repete jusqu'a trouver un catch

C'est pour ca que RAII fonctionne si bien avec les exceptions !

Les smart pointers, les fichiers RAII, etc. sont automatiquement nettoyes pendant le stack unwinding.

6.2 Exemple avec ressources multiples

```
1 void traiterFichiers() {
2     std::unique_ptr<int> ptr1 = std::make_unique<int>(10);
3     std::unique_ptr<int> ptr2 = std::make_unique<int>(20);
4     std::unique_ptr<int> ptr3 = std::make_unique<int>(30);
5
6     std::ifstream fichier("data.txt");
7
8     // Si une exception est lancee ici :
9     throw std::runtime_error("Erreur");
10
11     // ptr3, ptr2, ptr1 et fichier sont automatiquement detruits
12     // dans l'ordre inverse de creation
13     // AUCUNE FUITE
14 }
```

7 Relancer une exception

7.1 throw; sans argument

```
1 void fonction() {
2     try {
3         operationRisquee();
4     } catch (const std::exception& e) {
5         std::cout << "Log : " << e.what() << "\n";
6         throw; // Relance la MEME exception
7     }
8 }
9
10 int main() {
11     try {
12         fonction();
13     } catch (const std::exception& e) {
14         std::cout << "Attrapee au niveau main : " << e.what() << "\n";
15     }
16 }
```

throw; (sans argument) relance l'exception en cours. Utile pour :

- Logger une erreur puis la propager
- Faire un traitement partiel puis laisser un niveau superieur gerer
- Nettoyer des ressources puis re-lancer

7.2 Wrapper une exception

```
1 class HighLevelException : public std::exception {
2     std::string message;
3 public:
4     HighLevelException(const std::string& msg) : message(msg) {}
5     const char* what() const noexcept override {
6         return message.c_str();
7     }
8 };
9
10 void fonctionBasNiveau() {
11     throw std::runtime_error("Erreur bas niveau");
12 }
13
14 void fonctionHautNiveau() {
15     try {
16         fonctionBasNiveau();
17     } catch (const std::exception& e) {
18         // Wrapper l'exception bas niveau dans une exception haut niveau
19         throw HighLevelException(
20             "Erreur haut niveau causee par : " +
21             std::string(e.what())
22         );
23     }
24 }
```

8 Exercices pratiques

8.1 Exercice 1 : Validation avec exceptions (facile)

Enonce

Creez une fonction `validateEmail(string)` qui :

- Lance `invalid_argument` si l'email est vide
- Lance `invalid_argument` si l'email ne contient pas '@'
- Lance `invalid_argument` si l'email ne contient pas '.'
- Retourne `true` si tout est OK

Testez avec plusieurs cas dans un try-catch.

Solution :

```
1 bool validateEmail(const std::string& email) {
2     if (email.empty()) {
3         throw std::invalid_argument("Email vide");
4     }
5     if (email.find('@') == std::string::npos) {
6         throw std::invalid_argument("Email sans @");
7     }
8     if (email.find('.') == std::string::npos) {
9         throw std::invalid_argument("Email sans .");
10    }
11    return true;
12 }
13
14 int main() {
15     std::vector<std::string> emails = {
16         "test@example.com",
17         "",
18         "test.example.com",
19         "test@example"
20     };
21
22     for (const auto& email : emails) {
23         try {
24             validateEmail(email);
25             std::cout << email << " : valide\n";
26         } catch (const std::invalid_argument& e) {
27             std::cout << email << " : invalide - "
28                 << e.what() << "\n";
29         }
30     }
31 }
```

8.2 Exercice 2 : Exception personnalisée (moyen)

Enoncé

Creez une hierarchie d'exceptions pour une banque :

- `BankException` : classe de base
- `InsufficientFundsException` : solde insuffisant
- `InvalidAmountException` : montant invalide (negatif ou zero)
- `AccountLockedException` : compte bloque

Creez une classe `Account` avec `withdraw(montant)` qui utilise ces exceptions.

Solution :

```

1 class BankException : public std::exception {
2     protected:
3         std::string message;
4     public:
5         BankException(const std::string& msg) : message(msg) {}
6         const char* what() const noexcept override {
7             return message.c_str();
8         }
9 };
10
11 class InsufficientFundsException : public BankException {
12     public:
13         InsufficientFundsException(double needed, double available)
14             : BankException("Solde insuffisant: besoin de " +
15                             std::to_string(needed) +
16                             ", disponible: " +
17                             std::to_string(available)) {}
18 };
19
20 class InvalidAmountException : public BankException {
21     public:
22         InvalidAmountException()
23             : BankException("Montant invalide") {}
24 };
25
26 class AccountLockedException : public BankException {
27     public:
28         AccountLockedException()
29             : BankException("Compte bloque") {}
30 };
31
32 class Account {
33     private:
34         double balance;
35         bool locked;
36
37     public:
38         Account(double initial) : balance(initial), locked(false) {}
39
40         void withdraw(double amount) {
41             if (locked) {
42                 throw AccountLockedException();
43             }
44             if (amount <= 0) {

```

```

45         throw InvalidAmountException();
46     }
47     if (amount > balance) {
48         throw InsufficientFundsException(amount, balance);
49     }
50     balance -= amount;
51 }
52
53 double getBalance() const { return balance; }
54 void lock() { locked = true; }
55 };
56
57 int main() {
58     Account acc(100.0);
59
60     try {
61         acc.withdraw(50);    // OK
62         acc.withdraw(-10);   // Exception
63     } catch (const InvalidAmountException& e) {
64         std::cout << "Erreur : " << e.what() << "\n";
65     }
66
67     try {
68         acc.withdraw(200);   // Exception
69     } catch (const InsufficientFundsException& e) {
70         std::cout << "Erreur : " << e.what() << "\n";
71     }
72 }

```

8.3 Exercice 3 : RAII avec exceptions (difficile)

Enonce

Creez une classe Transaction qui :

- Commence une transaction dans le constructeur
- Commit dans commit()
- Rollback dans le destructeur si pas commit
- Si une exception est lancee, fait automatiquement rollback

Testez avec du code qui throw au milieu de la transaction.

Solution :

```

1  class Transaction {
2  private:
3      bool committed;
4      std::string name;
5
6  public:
7      Transaction(const std::string& n) : committed(false), name(n) {
8          std::cout << "Transaction " << name << " commencee\n";
9      }
10
11     ~Transaction() {
12         if (!committed) {
13             std::cout << "Transaction " << name

```

```
14         << " annulee (rollback)\n";
15     }
16 }
17
18 void commit() {
19     std::cout << "Transaction " << name << " validee (commit)\n";
20     committed = true;
21 }
22 };
23
24 void operationAvecSucces() {
25     Transaction t("Success");
26     std::cout << "Operation 1\n";
27     std::cout << "Operation 2\n";
28     t.commit(); // Tout s'est bien passe
29 }
30
31 void operationAvecErreur() {
32     Transaction t("Error");
33     std::cout << "Operation 1\n";
34     throw std::runtime_error("Erreur !");
35     t.commit(); // Jamais execute
36 }
37
38 int main() {
39     operationAvecSucces();
40
41     try {
42         operationAvecErreur();
43     } catch (const std::exception& e) {
44         std::cout << "Exception : " << e.what() << "\n";
45     }
46 }
47
48 // Affiche :
49 // Transaction Success commencee
50 // Operation 1
51 // Operation 2
52 // Transaction Success validee (commit)
53 // Transaction Error commencee
54 // Operation 1
55 // Transaction Error annulee (rollback)
56 // Exception : Erreur !
```

9 Patterns avec exceptions

9.1 Exception Guard

Protéger une section de code et garantir le nettoyage :

```

1  template<typename Func>
2  class ExceptionGuard {
3  private:
4      Func cleanup;
5      bool dismissed;
6
7  public:
8      ExceptionGuard(Func f) : cleanup(f), dismissed(false) {}
9
10     ~ExceptionGuard() {
11         if (!dismissed) {
12             try {
13                 cleanup();
14             } catch (...) {
15                 // Absorber les exceptions du cleanup
16             }
17         }
18     }
19
20     void dismiss() { dismissed = true; }
21 };
22
23 void exemple() {
24     int* ptr = new int[100];
25     ExceptionGuard guard([&]() { delete[] ptr; });
26
27     // Code qui peut throw
28     traiterDonnees();
29
30     guard.dismiss(); // Tout s'est bien passe
31     delete[] ptr;
32 }

```

9.2 Try-Catch dans les constructeurs

```

1  class Ressource {
2  private:
3      std::unique_ptr<int> data1;
4      std::unique_ptr<int> data2;
5
6  public:
7      Ressource() try
8          : data1(std::make_unique<int>(10)),
9            data2(std::make_unique<int>(20)) {
10
11         // Corps du constructeur
12         if (/* condition */) {
13             throw std::runtime_error("Erreur init");
14         }
15
16     } catch (const std::exception& e) {
17         std::cout << "Erreur construction : " << e.what() << "\n";

```

```

18         throw; // Obligatoire de relancer
19     }
20 };

```

9.3 Error Codes vs Exceptions

```

1  // Style ancien : codes d'erreur
2  enum class ErrorCode {
3      SUCCESS,
4      FILE_NOT_FOUND,
5      PERMISSION_DENIED,
6      INVALID_FORMAT
7  };
8
9  ErrorCode openFile(const std::string& name, File& out) {
10     // ...
11     return ErrorCode::SUCCESS;
12 }
13
14 // Style moderne : exceptions
15 File openFile(const std::string& name) {
16     // ...
17     if (error) {
18         throw FileException("Cannot open file");
19     }
20     return file;
21 }

```

Quand utiliser quoi ?

Exceptions :

- Erreurs exceptionnelles (pas le flux normal)
- Erreurs qui doivent être traitées
- Propagation sur plusieurs niveaux

Codes d'erreur :

- Performance critique
- Erreurs attendues et fréquentes
- Interfacage avec du code C

std::optional/std::expected (C++23) :

- Quand une valeur peut ne pas exister (pas une erreur)
- Alternative légère aux exceptions

10 Pieges et bonnes pratiques

10.1 Piege 1 : Catch par valeur

MAUVAIS

```

1 try {
2     // ...
3 } catch (std::exception e) { // SANS const&
4     // L'exception est COPIEE
5     // Probleme de slicing si exception derivee
6 }

```

BON

```

1 try {
2     // ...
3 } catch (const std::exception& e) { // AVEC const&
4     // Pas de copie, pas de slicing
5 }

```

10.2 Piege 2 : Lancer par pointeur

MAUVAIS

```

1 throw new std::runtime_error("Erreur"); // Pointeur !
2 // Qui va delete ?

```

BON

```

1 throw std::runtime_error("Erreur"); // Objet !
2 // Gere automatiquement

```

10.3 Bonne pratique 1 : Exception dans les templates

```

1 template<typename T>
2 class Stack {
3 public:
4     T pop() {
5         if (empty()) {
6             throw std::out_of_range("Stack vide");
7         }
8         // ...
9     }
10 };

```

10.4 Bonne pratique 2 : noexcept quand garanti

```

1 class Widget {
2 public:

```

```
3   Widget(Widget&& other) noexcept { /* ... */ }
4   Widget& operator=(Widget&& other) noexcept { /* ... */ }
5   void swap(Widget& other) noexcept { /* ... */ }
6 };
```

Pourquoi ? Les conteneurs STL peuvent optimiser s'ils savent que move ne throw pas.

11 Quiz de validation

11.1 Questions

1. Quelle est la difference entre throw et return ?
2. Que fait catch(...) ?
3. Pourquoi attraper par const reference ?
4. Qu'est-ce que le stack unwinding ?
5. Pourquoi un destructeur ne doit jamais lancer d'exception ?
6. Que fait throw ; sans argument ?
7. Quelle est la classe de base de toutes les exceptions standard ?
8. Quelle est la difference entre logic_error et runtime_error ?
9. Qu'est-ce que la strong exception guarantee ?
10. Pourquoi marquer une fonction noexcept ?

11.2 Reponses

1. throw interrompt le flux et cherche un catch. return retourne normalement a l'appelant.
2. Attrape TOUTES les exceptions, meme non-standard.
3. Pour eviter la copie et le slicing. Performance + securite.
4. Le processus de deroulement de la pile d'appels, detruisant tous les objets locaux jusqu'a trouver un catch.
5. Car si une exception est deja en cours, ca causerait une double exception et std::terminate().
6. Relance l'exception en cours. Utile pour logger puis propager.
7. std::exception avec sa methode virtuelle what().
8. logic_error : bugs/erreurs de logique. runtime_error : erreurs d'environnement.
9. Si exception, l'etat est identique a avant l'appel. Comme si rien ne s'etait passe.
10. Pour indiquer au compilateur qu'une fonction ne lance jamais d'exception. Permet des optimisations.

Fin de la Seance 6

Prochaine seance : Move Semantics et Rvalue References